Sprint 9.1 — Product Stabilization

UI/UX & User Journey Audit Report

12	17	22	15
Critical	High	Medium	Low

Feature Freeze | Audit Only | No Fixes Applied

July 2026

Executive Summary

This report documents the complete Sprint 9.1 audit of the DeepMindQ Enterprise Intelligence Operating System. The audit covers all 8 product phases (Phase 1: CRM Foundation through Phase 8: AI Revenue Copilot), spanning 212 API endpoints, 63 UI screen components, Prisma database schema (1,845 lines), authentication/security infrastructure, and end-to-end user journeys. This is an audit-only sprint — no fixes have been applied. All findings are categorized by severity for review and approval before Sprint 9.2 (Product Hardening).

The audit identified **66 total issues**: 12 Critical, 17 High, 22 Medium, and 15 Low. The most significant cluster is in Security (6 Critical issues), followed by Database (4 Critical) and API Architecture (2 Critical). The highest-priority action items are: (1) adding authentication to 190+ unprotected API endpoints, (2) fixing OTP brute-force vulnerability, (3) removing debug endpoints from production, (4) rotating leaked API keys, and (5) adding database connection pooling for Vercel serverless deployment. UI/UX issues are generally less severe, with the main concerns being 35 screens missing empty states and 9 locations where errors are silently swallowed.

Category	Critical	High	Medium	Low	Total
Security & Auth	6	8	3		17
Database & Schema	4	5	5	1	15
API Architecture	2	2	3		7
UI/UX & Components		2	7	10	19
Performance & N+1			3		3
User Journey Gaps			1	4	5
TOTAL	**12**	**17**	**22**	**15**	**66**

1. Security & Authentication

The security audit identified 6 Critical and 8 High severity issues. The most alarming finding is that **194 out of 198 API handler files** (all non-auth endpoints) have zero authentication checks. Anyone who can reach the API can read, modify, or delete all CRM data, trigger email sends, execute AI operations, and access administrative functions without any session verification. This is the single highest-priority fix that must be addressed in Sprint 9.2 before any production release.

C1 [CRITICAL] Missing Authentication on 190+ API Endpoints

Only 4 of 198 API handler files import requireAuth or getCurrentSession. All endpoints in g-crm, g-ai, g-data, g-outreach, g-strategy, g-system, g-intelligence, g-revenue-intelligence, g-ai-copilot, g-intel-acquisition have zero session verification. Any unauthenticated HTTP request can read/write the entire database.

`Files: src/app/api/g-crm/[...slug]/*.ts, src/app/api/g-data/[...slug]/*.ts, src/app/api/g-ai/[...slug]/*.ts, + 6 more groups`

C2 [CRITICAL] OTP Brute-Force Vulnerability — Attempts Never Increment on Wrong Codes

verifyOtp() queries DB with WHERE code=X. If code is wrong, findFirst returns null and 'Invalid code' is returned WITHOUT incrementing the attempts counter. MAX_ATTEMPTS=5 only triggers if someone already knows the correct code. An attacker can try all 1,000,000 possible 6-digit OTPs with zero lockout.

`Files: src/lib/otp.ts:241-253`

C3 [CRITICAL] Debug Endpoint Exposes API Key Fragments

GET /api/debug/env-check returns the first 8 characters of GEMINI_API_KEY and TAVILY_API_KEY. No authentication required. The endpoint also makes live API calls to verify the keys. Deployed and reachable in production.

`Files: src/app/api/debug/env-check/route.ts:8-10`

C4 [CRITICAL] Real API Keys in .env.vercel-pull

Plaintext GEMINI_API_KEY and TAVILY_API_KEY stored in .env.vercel-pull. While .gitignore excludes .env* files, the keys exist on disk and are visible to any process with filesystem access. Keys should be rotated.

`Files: .env.vercel-pull:7,11`

C5 [CRITICAL] Full Vercel OIDC Tokens in Env Files

VERCEL_OIDC_TOKEN with owner-level scopes stored in plaintext in .env.local and .env.vercel-pull. These tokens grant full project access on Vercel.

`Files: .env.local:2, .env.vercel-pull:12`

C6 [CRITICAL] Unprotected Seed Endpoint

POST /api/seed recreates entire database with sample data. No authentication check. Anyone can trigger a re-seed, potentially wiping production data.

`Files: src/app/api/g-system/[...slug]/seed.ts:4`

H1 [HIGH] CSRF Bypassed in Development Mode

If NODE_ENV !== 'production' and no CSRF token is present, request is allowed through. Zero CSRF protection in development. Staging environments running in non-production mode are fully exploitable.

`Files: src/lib/csrf.ts:18-21`

H2 [HIGH] Auth Bypassed on Exception in API Middleware

If getCurrentSession() throws (not returns null), catch block allows request with authorized:true and userId:'dev-user' in non-production. Dangerous fallback that could mask real auth issues.

`Files: src/lib/api-middleware.ts:65-68`

H3 [HIGH] Hardcoded Default NEXTAUTH_SECRET

NEXTAUTH_SECRET defaults to 'deepmindq-dev-secret-change-in-production'. If not overridden in production, all session signing uses a publicly known secret.

`Files: src/lib/validate-env.ts:7`

H4 [HIGH] Hardcoded Default UNSUBSCRIBE_SECRET

UNSUBSCRIBE_SECRET defaults to 'deepmindq-unsubscribe-secret-key'. Attacker can forge unsubscribe tokens for any email.

`Files: src/lib/unsubscribe.ts:11`

H5 [HIGH] OTP Code Leaked in Email Subject Line

Email subject includes the 6-digit OTP code: '${purposeLabel(purpose)} - ${code}'. Visible in notification previews, client sidebars, and server logs.

`Files: src/lib/otp.ts:190`

H6 [HIGH] OTP Code Returned to Client When Email Fails

When EMAIL_API_KEY is not configured, OTP code is returned as devCode in API response. If this fallback reaches production (missing env var), OTP codes are plaintext in responses.

`Files: src/lib/otp.ts:211-213, auth__request-otp.ts:43`

H7 [HIGH] Webhook Signature Verification Has Silent Bypass

If RESEND_WEBHOOK_SECRET is set but no signature header is present, webhook is still processed. Attacker can omit the signature header entirely to bypass verification.

`Files: src/app/api/g-outreach/[...slug]/webhooks__reply.ts:234-248, webhooks__bounce.ts:140-153`

H8 [HIGH] Password Change Deletes Current Session (Self-Lockout)

After password change, deleteMany({where:{userId}}) deletes ALL sessions including current. Comment says 'Don't delete current session' but WHERE clause has no exclusion. User gets locked out.

`Files: src/app/api/g-auth/[...slug]/auth__change-password.ts:51-56`

M1 [MEDIUM] No CORS Configuration

No Access-Control-Allow-Origin headers anywhere. Defaults to same-origin (safe), but API rewrites in next.config.ts may interact oddly with reverse proxies.

`Files: src/middleware.ts, next.config.ts`

M2 [MEDIUM] CSP Allows unsafe-eval and unsafe-inline

script-src 'self' 'unsafe-eval' 'unsafe-inline' negates XSS protection. Any injected script can execute.

`Files: src/middleware.ts:42`

M3 [MEDIUM] CSP connect-src Allows Any HTTPS Origin

connect-src 'self' https: allows data exfiltration to any HTTPS URL if XSS is achieved.

`Files: src/middleware.ts:47`

2. Database & Schema

The database audit examined the 1,845-line Prisma schema, 216 files using the Prisma client, and 4 files using transactions. Key findings include: zero Prisma enum types (30+ fields are free-text String), no migrations directory (schema managed via db push only, no rollback capability), missing connection pooling for Vercel serverless, and critical N+1 query patterns in the dedup endpoint that loads ALL contacts and performs O(n-squared) comparisons. Only 4 out of 216 DB-using files use transactions, leaving most multi-step operations vulnerable to partial failure and data corruption.

C7 [CRITICAL] Zero Prisma Enums — All Status Fields Are Free-Text String

30+ fields representing fixed enumerations (Contact.status, Company.status, User.role, etc.) are plain String type with comments listing allowed values. No DB-level validation. Any typo silently inserted. Examples: Contact.status has 11 valid values, Company.signalType has 8, SequenceEnrollment.status has 4. No enum constraints exist anywhere.

`Files: prisma/schema.prisma (30+ fields across entire file)`

C8 [CRITICAL] No Migrations Directory — No Schema Evolution Tracking

prisma/ directory contains only schema.prisma. No migrations/ folder exists. Database was created via 'prisma db push' which is not migration-safe and has no rollback capability. No deployment pipeline integration. Schema drift between environments is likely.

`Files: prisma/ (missing migrations/ directory)`

C9 [CRITICAL] Unbounded findMany() + O(n^2) Dedup Algorithm

leads__dedup.ts loads EVERY contact in database with no limit, then runs nested O(n^2) loop comparing every contact against every other. With 10,000 contacts: loads 10K rows, performs ~50M comparisons. Will cause memory exhaustion, request timeouts, and database overload.

`Files: src/app/api/g-crm/[...slug]/leads__dedup.ts:27`

C10 [CRITICAL] Only 4 of 216 Files Use Transactions

Vast majority of multi-step write operations are NOT transactional. Critical non-transactional operations include: email-worker (send + update queue + update draft + update contact), webhooks (create reply + update contact + create event), leads dedup merge (move drafts + move replies + mark duplicate), and companies bulk delete (cascades to 20+ related tables).

`Files: src/app/api/g-outreach/[...slug]/email-worker.ts, webhooks__reply.ts, leads__dedup.ts`

H9 [HIGH] Missing @unique on Contact.email

Contact.email has an index but no @unique constraint. Multiple contacts can have the same email. Dedup endpoint runs O(n^2) scan in application code. A unique constraint at DB level would prevent this at the source.

`Files: prisma/schema.prisma:17`

H10 [HIGH] No Serverless Connection Pooling (Neon/Vercel)

PrismaClient instantiated with only logging options. For Neon PostgreSQL on Vercel serverless, each cold start creates a new TCP connection. No @prisma/adapter-neon, no pgbouncer, no connection_limit. Will hit Neon connection limits under load.

`Files: src/lib/db.ts:11-18`

H11 [HIGH] Missing onDelete on 6 Relations

Six relations lack onDelete policy: Contact->ImportBatch, OpportunityRecommendation->CompanySignal, OpportunityRecommendation->SignalCapabilityMatch, EmailSequence->OpportunityRecommendation, Draft->ABTest, AIEngagementStrategy->StrategicInsight. Deleting parent records will fail if children reference them.

`Files: prisma/schema.prisma:59, 1171, 1172, 448, 522, 1802`

H12 [HIGH] Unbounded fetchDBMeta() Loads All Contacts + Companies

leads.ts loads ALL contacts and ALL companies just to build filter dropdown metadata. Should use groupBy aggregation queries. With large datasets, this causes unnecessary memory and CPU usage.

`Files: src/app/api/g-crm/[...slug]/leads.ts:281-283`

H13 [HIGH] N+1 in autoGenerateAlerts() — 100+ Individual DB Calls

Four separate for loops each executing findFirst + create per item. With 50 degraded sources, makes 100+ individual queries. Should batch with bulk operations.

`Files: src/lib/intelligence-sources/intelligence-alerts.ts:438-640`

H14 [HIGH] JSON Stored as String Instead of Json Type

10+ fields storing JSON are typed as String rather than Prisma Json type (Contact.enrichmentData, Company.tags, CompanyResearchCard.keyPeople, Job.payload, etc.). No DB-level JSON validation, no JSON query operators, manual JSON.parse/stringify in 100+ locations.

`Files: prisma/schema.prisma (multiple fields)`

M4 [MEDIUM] In-Memory Rate Limiter Ineffective in Production

Rate limits use Map in process memory. On Vercel serverless, each invocation gets fresh process. Rate limiting is non-functional in production.

`Files: src/lib/rate-limit.ts:18-19`

M5 [MEDIUM] Inconsistent Soft-Delete Pattern

Multiple overlapping deletion mechanisms: Contact.isSuppressed (Boolean), Contact.status 'suppressed'/'archived'/'duplicate', separate Suppression table. Contradictory states possible: isSuppressed:true AND status:'sent'.

`Files: Multiple models across schema`

M6 [MEDIUM] N+1 in retryAllFailed() — Sequential Updates in Loop

Loads all failed jobs then updates each individually. Should use single updateMany with where:{id:{in:retryableIds}}.

`Files: src/lib/workflow-engine/queue.ts:432-451`

M7 [MEDIUM] N+1 in email-worker — Sequential Per-Item Updates

3 sequential updates per item after email send (queue, draft, contact). With 50 items = 150 sequential writes. Not wrapped in transaction — partial failures leave inconsistent state.

`Files: src/app/api/g-outreach/[...slug]/email-worker.ts:69-158`

M8 [MEDIUM] Missing Index on Contact.normalizedName

Contact.normalizedName has no index. If contacts are searched by normalized name, will be full table scan.

`Files: prisma/schema.prisma:15`

M9 [MEDIUM] Company.normalizedName Not Unique

normalizedName has index but no unique constraint. Multiple companies with same normalized name can exist, causing dedup confusion.

`Files: prisma/schema.prisma:81`

L1 [LOW] AccountStrategy.companyId Optional When Semantically Required

AccountStrategy.companyId is String? but strategies are inherently company-specific. Should be required.

`Files: prisma/schema.prisma:743`

3. API Architecture & Error Handling

The API audit examined all 212 TypeScript files in the API directory. Beyond the critical auth gap (C1), the most concerning findings are: 8 files with silent .catch(() => {}) blocks that swallow errors without user feedback, and massive numbers of unbounded findMany() queries across 60+ endpoint files that will cause performance degradation with growing data volumes. Many list endpoints also lack pagination.

C11 [CRITICAL] 190+ API Endpoints Without Input Validation

Beyond the auth gap, many endpoints accept raw request bodies without Zod validation. While g-crm endpoints use validateBody, endpoints in g-ai, g-intelligence, g-outreach, g-strategy often parse JSON directly without schema validation. Malformed input can cause unhandled exceptions.

`Files: src/app/api/g-ai/[...slug]/*.ts, src/app/api/g-intelligence/[...slug]/*.ts`

C12 [CRITICAL] 60+ Unbounded findMany() Queries Without Pagination

Over 60 endpoint files use findMany() without .take() or pagination. Examples: queue.ts loads all send queue items, replies.ts loads all replies, sequences.ts loads all sequences. With growing data, these queries will cause memory exhaustion and slow responses.

`Files: src/app/api/g-outreach/[...slug]/queue.ts, replies.ts, sequences.ts, + 57 more files`

H15 [HIGH] 9 Silent .catch(() => {}) Blocks Swallow Errors

8 files catch errors and do nothing: page.tsx (3 locations), bounces-screen.tsx (2), queue-screen.tsx, tag-manager.tsx, custom-field-renderer.tsx, conversation-studio-screen.tsx, leads-screen.tsx, drafts-screen.tsx, replies-screen.tsx. Users see stale/empty data with no error indication.

`Files: src/app/page.tsx:480,902,993, src/components/screens/bounces-screen.tsx:64,77`

H16 [HIGH] Mock Reset-Password Returns Success Without Action

auth__reset-password__confirm.ts always returns {success:true} without doing anything. User believes password was reset when it wasn't.

`Files: src/app/api/g-auth/[...slug]/auth__reset-password__confirm.ts:6`

M10 [MEDIUM] Many fetch() Calls Don't Check res.ok

Screens like segments-screen.tsx call res.json() without checking if response was successful. A 500 error produces unhandled JSON parse error instead of meaningful error message.

`Files: Multiple screen components`

M11 [MEDIUM] No Pagination on Most List Endpoints

Most list endpoints return full result sets without cursor/offset pagination. With growing data, response sizes will increase unbounded, causing slow API responses and high memory usage.

`Files: Majority of GET endpoints`

M12 [MEDIUM] Inconsistent Error Response Format

Some endpoints return {error:'message'}, others return {message:'...'}. No standard error envelope. Frontend error handling must handle multiple formats.

`Files: Various API endpoints`

4. UI/UX & Component Audit

The UI audit covered 63 screen components, the monolithic page.tsx (1,035 lines), 11 shared components, and ~50 UI primitives. The application uses a hash-based SPA routing pattern with lazy-loaded screens. Loading states are generally good (43/63 screens have skeletons), but empty states are missing from 35 screens, and error handling has gaps with 9 silent error catches. The main structural concern is the monolithic page.tsx and inconsistent use of inline styles (835+ instances) instead of the CSS variable system.

H17 [HIGH] page.tsx is a 1,035-Line Monolith

All navigation config, 14 bridge wrappers, 50+ screen map entries, ScreenErrorBoundary, ScreenLoader, inline AppShell, and HomePage in one file. Should be decomposed into: nav-config.ts, screen-map.ts, screen-loader.tsx, app-shell.tsx.

`Files: src/app/page.tsx (entire file)`

H18 [HIGH] 35 Screens Missing EmptyState Component

Only 28 of 63 screens use EmptyState. Missing from: pipeline-screen, audit-logs-screen, intelligence-timeline, intelligence-inbox, intelligence-knowledge, company-resolution-modal, templates-screen, ai-usage-dashboard, revenue-intelligence screens, and more.

`Files: Multiple screens in src/components/screens/`

M13 [MEDIUM] 835+ Inline style={{}} Across 46 Screen Files

Massive use of inline styles instead of CSS variables and Tailwind utilities. Theme changes require editing dozens of files. Dark mode impossible. Worst offenders: settings-screen.tsx (60), command-center-screen.tsx (174), companies-screen.tsx (67).

`Files: Multiple screen files`

M14 [MEDIUM] Dead app-shell.tsx File — Never Imported

Complete AppShell with Sidebar/Header exists in app-shell.tsx but is never imported. Real app shell is defined inline in page.tsx. Different nav structure (flat 12 items vs 8 collapsible sections with 50+ items).

`Files: src/components/app-shell.tsx (entire file, 379 lines)`

M15 [MEDIUM] Duplicate EmptyState Components with Incompatible Props

Two EmptyState components exist: design-system.tsx (6 props, amber-600) and animated-components.tsx (4 props, motion.div, gold). 28 screens import from animated-components, others from design-system. Inconsistent appearance.

`Files: src/components/shared/design-system.tsx:24, src/components/ui/animated-components.tsx:366`

M16 [MEDIUM] 5 Different Hardcoded Gold Hex Values

#B8860B, #D4AF37, #D4A843, #c9a84c, #8B6914 used across 5+ screens instead of CSS variable --color-gold.

`Files: dashboard-screen.tsx:10, companies-screen.tsx:27, command-center-screen.tsx:52, + 3 more`

M17 [MEDIUM] Intelligence Health Stat Cards Not Responsive

grid-cols-4 with no responsive breakpoint. Cards overflow on tablets/narrow viewports. Should be grid-cols-2 lg:grid-cols-4.

`Files: src/components/screens/intelligence-health-screen.tsx:182`

M18 [MEDIUM] Hash-Based Routing Limitations

window.location.hash for routing has limitations: deep linking loses scroll position, company detail doesn't update hash for company ID, no server-side rendering.

`Files: src/app/page.tsx:429-448`

M19 [MEDIUM] Duplicate Nav Key 'opportunity-radar'

Same key in both 'REVENUE INTELLIGENCE' and 'INTELLIGENCE LAYER' sections. Clicking one updates both. Confusing navigation.

`Files: src/app/page.tsx:119 and 136`

M20 [MEDIUM] No Back Button on Most Detail Views

Company Detail has onBack prop, but most detail overlays (contact, segment, draft) rely only on breadcrumb. On mobile, truncated breadcrumbs leave users stuck.

`Files: Multiple detail screen components`

M21 [MEDIUM] Segment Create Dialog — Minimal Validation

Only checks segment name. Score range accepts negative values. No validation that minScore < maxScore.

`Files: src/components/screens/segments-screen.tsx:90`

M22 [MEDIUM] Leads Inline Editing Has No Validation Feedback

Multiple inline edit dialogs for lead data lack inline validation. Errors shown only as toast notifications after API submission fails.

`Files: src/components/screens/leads-screen.tsx`

L2 [LOW] not-found.tsx Uses Dark Theme, App is Light

Uses hardcoded dark colors while the app is light-themed. Jarring visual transition for 404 pages.

`Files: src/app/not-found.tsx:6-15`

L3 [LOW] loading.tsx Light Theme vs Login Dark Theme

loading.tsx shows light spinner on #FAFAFA, but auth check loading in page.tsx uses dark background #0a0c10.

`Files: src/app/loading.tsx:3, src/app/page.tsx:1014-1021`

L4 [LOW] Notification Bell Always Shows Indicator Dot

Always-visible gold dot regardless of actual notification count. Misleading to users.

`Files: src/app/page.tsx:817`

L5 [LOW] Sequence Create Dialog — Only Toast-Level Validation

toast.error('Name and all step subjects/bodies required') — generic toast, no inline field-level validation showing which step is missing content.

`Files: src/components/screens/sequences-screen.tsx:120-123`

L6 [LOW] Research Agent Has No Loading Skeleton

Shows phase text while loading but no skeleton of expected output structure.

`Files: src/components/screens/research-agent-screen.tsx:145-200`

L7 [LOW] Intelligence Health Loading State Uses Bare Spinner

Uses RefreshCw with animate-spin instead of project's Skeleton component. Inconsistent with other screens.

`Files: src/components/screens/intelligence-health-screen.tsx:122-129`

L8 [LOW] ScreenErrorBoundary Lacks componentDidCatch Logging

Errors caught but not logged to monitoring service. Compare with global ErrorBoundary which does log.

`Files: src/app/page.tsx:326-356`

L9 [LOW] Breadcrumbs Show Max 2 Levels

Only ['Companies','Company Detail']. No deep trail for sub-navigation paths like Revenue Intelligence > Company Brief > [Name].

`Files: src/app/page.tsx:527-533`

L10 [LOW] Oversized Screen Components

leads-screen.tsx (110KB), settings-screen.tsx (110KB), company-detail-screen.tsx (1588 lines), command-center-screen.tsx (~890 lines). Should be split into sub-components.

`Files: Multiple screen files`

L11 [LOW] Segment Detail Dialog Uses Plain Text for Empty State

Bare paragraph instead of project's EmptyState component with icon and action button.

`Files: src/components/screens/segments-screen.tsx:320`

5. Performance & N+1 Query Patterns

Performance issues are primarily driven by two patterns: (1) unbounded database queries loading entire tables into memory, and (2) N+1 query patterns in loops. The most critical is the dedup endpoint's O(n-squared) algorithm. Additionally, the absence of connection pooling for Vercel serverless means each cold start creates a new database connection, which will compound the query performance issues under load.

M23 [MEDIUM] N+1 in Bulk Tag Operations — Sequential Read-Modify-Write

companies__bulk.ts loops through companies and updates each individually with JSON field manipulation. Max 100 companies = 100 sequential updates. JSON parse/stringify per item compounds the issue.

`Files: src/app/api/g-crm/[...slug]/companies__bulk.ts:83-189`

M24 [MEDIUM] N+1 in Draft Merge During Dedup

leads__dedup.ts POST handler: for each secondary contact, executes findUnique + per-draft update + per-reply update + secondary update + primary update. With 10 secondaries with 5 drafts and 3 replies = 110 queries.

`Files: src/app/api/g-crm/[...slug]/leads__dedup.ts:124-178`

M25 [MEDIUM] N+1 in Webhook Reply findOriginalItem()

Loops through message IDs querying drafts one at a time. Should batch with where:{messageId:{in:messageIds}}.

`Files: src/app/api/g-outreach/[...slug]/webhooks__reply.ts:191-204`

6. User Journey Validation

End-to-end user journey validation identified gaps in the login flow and some navigation inconsistencies. The core authentication flow (email -> OTP -> session -> dashboard) functions correctly. However, the landing page's Sign In buttons were previously non-functional (fixed during this audit session). Several navigation patterns need attention for production readiness.

M26 [MEDIUM] Landing Page Sign In Flow Involves Page Reload

Sign In navigates from landing-page.html to /login.html (full page reload). After OTP verification, redirect to '/' triggers middleware rewrite back to landing-page.html if cookie not set before redirect. Potential loop risk.

`Files: public/landing-page.html:761, public/login.html, src/middleware.ts:16-19`

L12 [LOW] No 'Back to List' on Most Detail Views

Company Detail has onBack. Contact, segment, draft details rely on breadcrumb only. On mobile, users can get stuck.

`Files: Multiple detail screen components`

L13 [LOW] 404 Page Uses Different Theme Than App

Dark-themed 404 page while app is light-themed. Confusing transition when a logged-in user hits a 404.

`Files: src/app/not-found.tsx`

L14 [LOW] Notification 'View All Activity' Links to Audit Log

Clicking notification dropdown's 'View all activity' navigates to audit log, not a notifications page. Different concepts conflated.

`Files: src/app/page.tsx:872`

L15 [LOW] No Session Timeout Warning to User

Sessions auto-expire but no warning shown to user before session ends. User may lose unsaved work without notification.

`Files: src/lib/session.ts`

7. Positive Findings

Session tokens — Cryptographically random 256-bit tokens via crypto.getRandomValues

`session.ts:18`

Cookie security — httpOnly:true, secure in production, sameSite:'lax'

`session.ts:61-63`

Password hashing — PBKDF2-SHA256, 100k iterations, 128-bit salt, constant-time comparison

`password.ts`

Session rolling — Sessions auto-extend on each request

`session.ts:123-128`

User enumeration — Same error for wrong email vs wrong password with fixed delay

`auth__login.ts:42-43`

Loading states — 43 of 63 screens have proper skeleton loading states

`Multiple screens`

Toast notifications — 356 toast calls across 37 files with consistent Sonner config

`Multiple files`

Mobile sidebar — Well-implemented responsive sidebar with overlay backdrop

`page.tsx`

CSRF protection — Proper CSRF tokens on POST endpoints (when not in dev mode)

`csrf.ts`

Registration disabled — Single-owner workspace correctly blocks registration

`auth__register.ts`

Powered-by hidden — poweredByHeader:false removes X-Powered-By

`next.config.ts:11`

8. Sprint 9.2 Priority Recommendations

Based on the audit findings, the following priority order is recommended for Sprint 9.2 (Product Hardening). Issues should be addressed in this order, with all Critical items completed before moving to High, and all High completed before Medium/Low.

Priority	ID	Action	Area	Detail
P0	C1	Add requireAuth() to ALL 190+ API route handlers	Security	Addresses authentication gap on all non-auth endpoints
P0	C2	Fix OTP brute-force: separate lookup from code check	Security	Lookup by email+purpose, then compare code with attempt counting
P0	C3	Remove or protect /api/debug/env-check	Security	Delete endpoint or add auth + rate limiting
P0	C4/C5	Rotate all leaked API keys (Gemini, Tavily, Vercel OIDC)	Security	Generate new keys, update Vercel env vars, delete old keys
P0	C6	Protect /api/seed endpoint with auth	Security	Add requireAuth check
P1	H3/H4	Remove hardcoded secrets, require in production	Security	Fail startup if NEXTAUTH_SECRET/UNSUBSCRIBE_SECRET not set in prod
P1	H5/H6	Move OTP code to email body, remove devCode in prod	Security	Change subject to 'Your verification code is ready'
P1	H7	Fix webhook signature: reject if secret set but no sig	Security	Add check: if secret exists, signature header is required
P1	H8	Fix password change to exclude current session	Security	Add current session ID exclusion to deleteMany WHERE
P1	H10	Add Neon connection pooling for serverless	Database	Add @prisma/adapter-neon or pgbouncer config
P1	C9	Rewrite dedup with SQL GROUP BY instead of O(n^2)	Database	Use Prisma groupBy or raw SQL for matching
P1	C10	Add transactions to webhook handlers and email worker	Database	Wrap multi-step operations in $transaction
P2	C7	Convert top 10 String enums to Prisma enum types	Database	Start with Contact.status, Company.status, User.role
P2	C8	Initialize prisma migrations directory	Database	Create baseline migration from current schema
P2	H15	Replace silent .catch with error toasts/banners	UI/UX	Add toast.error() in all 9 catch blocks
P2	C12	Add pagination to all list endpoints	API	Implement cursor-based pagination
P2	H17	Decompose page.tsx monolith	Architecture	Extract nav-config, screen-map, app-shell
P2	H18	Add EmptyState to 35 screens missing it	UI/UX	Use animated-components EmptyState consistently
P3	M13	Replace hardcoded colors with CSS variables	UI/UX	Start with worst offenders (command-center, settings)
P3	M4	Replace in-memory rate limiter with Redis/Upstash	Performance	Required for production rate limiting
P3	H14	Convert JSON String fields to native Json type	Database	Contact.enrichmentData, Company.tags, etc.